CODERHOUSE

Proyecto Integrador Final

Data Analytics

Octubre 2026

Agustin Ezequiel Cuenca

Fundamentación del proyecto

Evaluación integradora final del programa de Data Analytics. Pone a prueba la capacidad de resolver un caso de negocio real aplicando todo lo aprendido: modelado de datos, consultas SQL, transformación y visualización con Power BI, y comunicación de hallazgos (storytelling con datos).

Los tres pilares de la integración

Para que un proyecto sea verdaderamente integrador, debe tejer una narrativa coherente a través de tres dimensiones técnicas:

A. La base de datos (extracción y transformación con SQL)
El proyecto comienza en las profundidades de los datos crudos. No se trata simplemente de escribir una consulta `SELECT *`. Se trata de entender la arquitectura de la información: ¿Cómo se relacionan las tablas de ventas con las de inventario? ¿Cómo filtramos los datos nulos que podrían sesgar nuestro análisis? SQL es aquí el cimiento del edificio; si el cimiento es débil o está mal construido, todo el análisis posterior carece de validez.

B. El análisis del "por qué" (estadística aplicada)
Una vez que tienes los datos, la estadística es la lente que te permite ver patrones. No basta con decir "las ventas subieron". Un proyecto integrador de nivel profesional responde: "¿Es esta subida estadísticamente significativa o es puro azar?", "¿Cuál es la correlación entre el gasto en marketing y la tasa de conversión?". Aquí aplicas medidas de tendencia central, dispersión y, potencialmente, análisis predictivos básicos para dar sustento científico a tus conclusiones.

C. La narrativa visual (storytelling con Power BI)
La etapa final es la traducción. Los directivos de una empresa rara vez quieren ver código SQL o fórmulas estadísticas complejas. Quieren ver insights (percepciones accionables). Power BI actúa como el puente comunicativo. La integración ocurre cuando el dashboard que construyes no solo es "bonito", sino que permite al usuario navegar por los datos que tú preparaste y validaste en las etapas anteriores.

Principios de una evaluación de calidad

Para que tu certificación tenga peso en el mercado, el proyecto integrador se rige por principios de evaluación profesional que aseguran su rigor:

- **Validez:** El proyecto debe medir lo que dice medir. Si el objetivo es evaluar a un analista de datos, el proyecto no debería centrarse únicamente en la estética del diseño, sino en la precisión de los datos y la relevancia de las conclusiones de negocio.

- **Confiabilidad:** Los criterios de evaluación (rúbricas) son claros y objetivos. Cualquier evaluador experto, al mirar tu trabajo bajo estos criterios, debería llegar a una conclusión similar sobre tu desempeño.

- **Transparencia:** Como estudiante, conoces de antemano qué se espera de ti. No hay "preguntas trampa"; el desafío reside en la complejidad del problema, no en la ambigüedad de las instrucciones.

Errores comunes y mejores prácticas

Incluso los estudiantes más técnicos pueden fallar en el proyecto integrador si no cuidan la visión de conjunto:

Error común	Por qué es un problema	Mejor práctica
Tratamiento de silos	Hacer un gran SQL pero que no responde a lo que luego muestras en Power BI.	Mantener una "hilo conductor" desde la primera consulta hasta el último gráfico.
Ignorar la limpieza de datos	Presentar análisis basados en datos duplicados o erróneos.	Documentar los pasos de limpieza y validar los resultados en cada etapa.
Complejidad innecesaria	Usar modelos estadísticos muy complejos que no se traducen en valor para el negocio.	Aplicar el principio de parsimonia: la explicación más sencilla que resuelva el problema suele ser la mejor.
Falta de contexto de negocio	Presentar números sin explicar qué significan para la empresa.	Siempre acompaña los datos con una recomendación: "¿Y ahora qué hacemos con esta información?".

Estructura de la evaluación

La evaluación final está compuesta por dos secciones principales:

1. **Preguntas teóricas**
   - Cubren conceptos clave del programa.
   - Enfocadas en recordar y analizar fundamentos de análisis de datos, bases de datos, SQL, modelado y visualización.

2. **Ejercicios prácticos**
   - Resolución de casos prácticos que requieren aplicar técnicas de consulta SQL, modelado de datos y creación de narrativas visuales.

Formato de entrega

La entrega del proyecto final deberá realizarse mediante **un único documento**.

Una vez finalizado el trabajo, deberás **integrar capturas de pantalla que demuestren lo desarrollado**, de modo que el evaluador pueda comprender todo el proceso de análisis mirando en documento sin necesidad de acceder a cada una de las herramientas utilizadas.

La presentación debe funcionar como **un reporte visual del análisis realizado**, combinando explicaciones, gráficos, resultados y consultas de SQL obtenidas desde la herramienta

Estructura recomendada de la presentación

La presentación debe organizarse como un reporte de análisis de datos, incluyendo las siguientes secciones:

1. Contexto del problema (pregunta de negocio)
2. Descripción del dataset y fuentes
3. Análisis exploratorio de datos (EDA)
4. Modelo de datos y consultas SQL
5. Transformación y limpieza (Power Query)
6. Dashboard y visualizaciones (Power BI)
7. Conclusiones e insights

Cada sección debe incluir **capturas relevantes de lo realizado, junto con una breve explicación de los resultados**.

Instrucciones

Sección 1: preguntas teóricas

1. ¿Cuál es la diferencia fundamental entre datos e información y por qué es importante para la toma de decisiones?

2. Enumera y describe brevemente los sublenguajes de SQL y su función principal.

3. Explica cómo la inteligencia artificial puede complementar el análisis de datos en un entorno empresarial.

4. ¿Qué principios básicos del storytelling aplicado a datos son esenciales para crear narrativas visuales efectivas?

Sección 2: ejercicios prácticos

1. Consulta SQL

Para resolver este ejercicio, trabajá con un dataset real que incluya información de **ventas y clientes** o que pueda adaptarse a ese escenario.

Selección del dataset
Elegí un dataset empresarial público disponible.

- El dataset debe permitir construir (o derivar) al menos dos tablas: **ventas** y **clientes**.
- Si el dataset viene en una sola tabla, deberás **separarlo o estructurarlo** para representar ambas entidades.

Preparación de datos (obligatorio)
Antes de escribir la consulta:

- Revisá el dataset y **limpiá los datos necesarios** (por ejemplo: valores vacíos, formatos de fecha, nombres de columnas).
- Conservá únicamente las columnas relevantes para este ejercicio, como mínimo:
  - Identificador o nombre del cliente
  - Fecha de compra
  - Total de la venta (o variable equivalente que permita calcularlo)
  - Asegurate de que la columna de fecha esté en un formato que permita filtrar “últimos 30 días” (contá los últimos 30 días a partir de la fecha que tenga la última venta del dataset).

Consulta SQL
Dado el conjunto de tablas de ventas y clientes (derivadas del dataset seleccionado), escribí una consulta SQL que:

1. Extraiga el **nombre del cliente**, la **fecha de la compra** y el **total de la venta**.
2. Filtre las ventas realizadas en los **últimos 30 días**.
3. Ordene los resultados por **fecha de compra descendente**.

Aclaración
En tu respuesta, incluí:

- La consulta SQL (print de pantalla).
- Una breve explicación de qué columnas y tablas utilizaste y por qué son adecuadas para cumplir la consigna.

2. Modelado de datos

Diseña un diagrama ER simple que represente una relación entre clientes, productos y ventas. Describe brevemente las entidades y relaciones.

3. Visualización y storytelling

Imagina que debes presentar un informe de ventas mensual a un equipo de marketing. Describe qué tipo de gráfico(s) usarías y cómo estructurarías la narrativa para que sea clara y persuasiva. Podes utilizar, Excel o Power BI.

Sección 1

1. Los **datos** son registros crudos de hechos sin procesar ni interpretar, como un número, una fecha, un nombre o un monto de venta aislado, por sí solos no comunican significado. Son la materia prima que se recopila a través de diversas fuentes, como sensores, encuestas o sistemas transaccionales.

En cambio, la **información** es el resultado de organizar, procesar y contextualizar esos datos para que respondan a una pregunta concreta. Por ejemplo, un monto de venta aislado es un dato, mientras que "las ventas del cliente cayeron un 40% respecto al trimestre anterior" es información. La diferencia está en el valor interpretativo y el contexto.

La información es importante para la **toma de decisiones** ya que reduce la incertidumbre, permite detectar tendencias actuando a tiempo en consecuencia, y a evitar conclusiones erróneas.

2. El lenguaje SQL se divide en sublenguajes según el tipo de operación que realizan sobre la base de datos:

- **DDL – Data Definition Language (Lenguaje de Definición de Datos)**
  Define y modifica la estructura de la base de datos: tablas, columnas, índices, restricciones y vistas.
  Comandos principales: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`

- **DML – Data Manipulation Language (Lenguaje de Manipulación de Datos)**
  Gestiona el contenido de las tablas: insertar, modificar o eliminar registros.
  Comandos principales: `INSERT`, `UPDATE`, `DELETE`

- **DQL – Data Query Language (Lenguaje de Consulta de Datos)**
  Consulta y recupera datos. Es el sublenguaje más usado en análisis de datos.
  Comandos principales: `SELECT`

- **DCL – Data Control Language (Lenguaje de Control de Datos)**
  Controla los permisos y accesos de los usuarios sobre los objetos de la base.
  Comandos principales: `GRANT`, `REVOKE`

- **TCL – Transaction Control Language (Lenguaje de Control de Transacciones)**
  Administra las transacciones, asegurando que un conjunto de operaciones se ejecute completo o no se ejecute (integridad de los datos).
  Comandos principales: `COMMIT`, `ROLLBACK`, `SAVEPOINT`

3. La **inteligencia artificial** puede **complementar** el análisis de datos en un entorno empresarial automatizando tareas repetitivas, detectando valores atípicos, registros duplicados, formatos inconsistentes o datos faltantes, y detectando patrones difíciles de ver a simple vista. Permite pronosticar ventas o demanda, estimar la probabilidad de que un cliente deje de comprar o anticipar quiebres de stock. También facilitan la segmentación de clientes en grupos con comportamientos similares, y la detección de anomalías como fraudes o errores en transacciones.

Por otro lado, herramientas como Copilot en Power BI u otros asistentes permiten generar consultas SQL, fórmulas DAX o visualizaciones a partir de preguntas en lenguaje natural. Esto acerca el análisis a usuarios no técnicos y agiliza el trabajo del propio analista.

Un ejemplo concreto es el de Netflix, que usa modelos de inteligencia artificial para analizar el historial de visualización de cada usuario y recomendarle contenido. Según la propia empresa, la mayor parte de lo que se mira en la plataforma llega a través de esas recomendaciones. En un entorno de ventas como el de este trabajo, la misma lógica permitiría anticipar qué productos ofrecerle a cada cliente según sus compras anteriores.

Sin embargo, es importante aclarar que la IA no reemplaza al analista de datos. Su uso tiene límites y depende completamente de la calidad de los datos con los que trabaja. Es decir, si la entrada es deficiente, los resultados también lo serán. Además, puede reproducir sesgos o generar respuestas incorrectas que

parecen convincentes, por lo que **siempre requiere validación humana y criterio de negocio**, especialmente cuando se manejan datos personales o decisiones sensibles.

4. El **storytelling** consiste en comunicar un mensaje a través de un relato. Aplicado a los datos, implica organizar los resultados de un análisis como una historia apoyada en visualizaciones, para que la audiencia entienda qué está pasando y qué decisión tomar, ya que un gráfico sin propósito es solo ruido visual. Esa historia debe girar en torno a un mensaje central, que es el insight más importante que surge del análisis y debería poder resumirse en una sola frase. Ese mensaje se construye pensando en la audiencia, porque no todos dentro de una organización necesitan saber lo mismo, y debe concluir con una llamada a la acción, es decir, una recomendación concreta sobre qué hacer, porque el objetivo del storytelling no es solo informar, sino ayudar a tomar decisiones.

   Para que ese mensaje sea claro y convincente, la narrativa necesita una estructura que guíe a la audiencia de principio a fin, igual que cualquier relato. Comienza con una introducción que presenta el contexto y la pregunta de negocio, de modo que el público entienda por qué el análisis es relevante. Continúa con un desarrollo, donde los datos, las comparaciones y las visualizaciones explican lo que está ocurriendo y permiten descubrir el hallazgo principal. Termina con una conclusión, que resume ese hallazgo y lo transforma en una acción concreta. Esta secuencia evita que la presentación se convierta en una sucesión de gráficos sueltos y hace que el mensaje sea más fácil de comprender, recordar y aplicar.

   Esa misma lógica se traslada al diseño visual. Es clave priorizar la información, destacando los KPIs más relevantes con color, tamaño y posición, y respetar la forma en que leemos, de izquierda a derecha y de arriba hacia abajo, siguiendo un patrón en "Z". Por eso lo más importante va arriba y el recorrido avanza de lo general al detalle. A su vez, cada gráfico debe elegirse según el objetivo: barras para comparar categorías, líneas para tendencias en el tiempo, dispersión para relacionar variables y tarjetas o KPIs para cifras clave. Conviene evitar las tortas con muchas categorías, los efectos 3D y los ejes mal configurados, porque distorsionan la lectura, y asegurar que la visualización sea accesible, con buen contraste de colores, tipografías legibles y etiquetas claras, pensando también en personas con daltonismo, por ejemplo.

Sección 2

1. **Consulta SQL**

   Aclaraciones

   - En cada parte, primero se describe el procedimiento y debajo se adjuntan las capturas de pantalla. Sobre cada captura se indica el archivo al que corresponde.
   - Los archivos `SQLQuery1.sql` a `SQLQuery4.sql` del repositorio son los trabajados en clase: crean la base de datos, cargan los registros y contienen consultas de práctica. Para este ejercicio se corrigió la carga de fechas en `SQLQuery2.sql`, se agregaron los clientes a la base de datos en `SQLQuery5.sql` y se realizó la consulta solicitada en `SQLQuery6.sql`.

   Selección del dataset

   El dataset analizado es el de Nike (`BD_Nike`), el mismo que se trabajó durante la cursada. Este dataset registra las ventas de productos de la marca en ciudades de Estados Unidos entre enero de 2020 y diciembre de 2024. En la base de datos está organizado como un modelo estrella.
   La tabla de hechos `FactNikeSales` guarda una fila por cada venta, con su fecha, el precio por unidad, las unidades vendidas, el costo unitario y el margen operativo. Esta tabla se relaciona con tablas de dimensión que describen el producto (nombre, categoría, subcategoría, marca y color, entre otras) y la ubicación de la venta (`DimRegion`, `DimState` y `DimCity`).

   La consigna pide trabajar con al menos dos tablas, una de ventas y otra de clientes. La tabla de ventas ya existe, pero el dataset no tiene información de clientes. Por eso, se decidió derivar una tabla de clientes a partir de los datos disponibles. El procedimiento se explica en el apartado del archivo `SQLQuery5.sql`.

Preparación de datos

Los datos originales están en archivos JSON y se cargan en la base con el script `SQLQuery2.sql`. Al revisar la carga de la tabla de ventas se detectó un error en el formato de la fecha. En el archivo, la columna `INVOICE DATE` tiene el formato día/mes/año (por ejemplo, 3/1/2020 es el 3 de enero), pero la conversión a fecha se hacía con el estilo `101`, que corresponde a mes/día/año. Como consecuencia, las fechas con un día menor o igual a 12 se cargaban con el día y el mes invertidos, y las demás no podían convertirse y quedaban afuera por el filtro del `WHERE`. La base tenía 90.270 ventas en lugar de las 229.471 del archivo.

Para corregirlo se cambió la conversión al estilo `103`, que corresponde a día/mes/año, y se volvió a cargar la tabla. Después de la corrección, la tabla tiene las 229.471 ventas del archivo, con fechas entre el 3 de enero de 2020 y el 30 de diciembre de 2024, y ninguna quedó sin fecha. La columna `InvoiceDate` quedó guardada con el tipo `DATE`, que es el formato que permite comparar fechas y filtrar los últimos 30 días.

En la misma carga se resuelven otros aspectos de la limpieza. Los nombres de columnas del archivo que tienen espacios, como `INVOICE DATE` o `Price per Unit`, se renombran sin espacios (`InvoiceDate`, `PricePerUnit`). Los valores decimales vienen con coma, así que con `REPLACE` se reemplaza la coma por un punto antes de convertirlos al tipo `DECIMAL` con `TRY_CONVERT`. Esta función devuelve un valor vacío (`NULL`) en lugar de un error cuando un dato no puede convertirse, y el `WHERE` final descarta las filas que no tengan una fecha válida. Por último, de todas las columnas disponibles, la consulta del ejercicio usa solo las necesarias: el nombre del cliente, la fecha de compra y las dos columnas que permiten calcular el total de la venta.

SQLQuery2.sql

```
214   -- FactNikeSales
215   DECLARE @jsonFact NVARCHAR(MAX);
216
217   SELECT @jsonFact = BulkColumn
218   FROM OPENROWSET(
219     BULK 'C:\data-json\ventas_nike.json',
220     SINGLE_CLOB
221   ) AS j;
222
223   ;INSERT INTO dbo.FactNikeSales (
224     InvoiceDate,
225     id_Region, id_State, id_City,
226     id_DEPARTMENT, id_Category, id_SubCategory,
227     id_PRODUCT_NAME, id_DESCRIPTION, id_PRODUCT_TYPE,
228     id_COLOR, id_BRAND, id_AVAILABILITY,
229     PRODUCT_SIZE,
230     PricePerUnit, UnitsSold, CostoUnitario, OperatingMargin
231   )
232   SELECT
233     -- las fechas del JSON vienen como dd/mm/yyyy (estilo 103)
234     TRY_CONVERT(DATE, InvoiceDate, 103),
235
236     id_Region, id_State, id_City,
237     id_Department, id_Category, id_SubCategory,
238     id_PRODUCT_NAME, id_DESCRIPTION, id_PRODUCT_TYPE,
239     id_Color, id_BRAND, id_AVAILABILITY,
240     PRODUCT_SIZE,
```

```
241
242     TRY_CONVERT(DECIMAL(12,2), REPLACE(PricePerUnit, ',', '.')),
243     UnitsSold,
244     TRY_CONVERT(DECIMAL(12,2), REPLACE(CostoUnitario, ',', '.')),
245     TRY_CONVERT(DECIMAL(6,3),  REPLACE(OperatingMargin, ',', '.'))
246   FROM OPENJSON(@jsonFact)
247   WITH (
248     InvoiceDate NVARCHAR(30) '$."INVOICE DATE"',
249
250     id_Region INT '$.id_Region',
251     id_State INT '$.id_State',
252     id_City INT '$.id_City',
253
254     id_Department INT '$.id_Department',
255     id_Category INT '$.id_Category',
256     id_SubCategory INT '$.id_SubCategory',
257
258     id_PRODUCT_NAME INT '$.id_PRODUCT_NAME',
259     id_DESCRIPTION INT '$.id_DESCRIPTION',
260     id_PRODUCT_TYPE INT '$.id_PRODUCT_TYPE',
261
262     id_Color INT '$.id_Color',
263     id_BRAND INT '$.id_BRAND',
264     id_AVAILABILITY INT '$.id_AVAILABILITY',
265
266     PRODUCT_SIZE NVARCHAR(40) '$.PRODUCT_SIZE',
267
268     PricePerUnit NVARCHAR(40) '$."Price per Unit"',
269     UnitsSold INT '$."Units Sold"',
270     CostoUnitario NVARCHAR(40) '$."Costo Unitario"',
271     OperatingMargin NVARCHAR(40) '$."Operating Margin"'
272   )
273   WHERE TRY_CONVERT(DATE, InvoiceDate, 103) IS NOT NULL;
274   GO
```

Consulta de verificación:

SQLQuery2.sql

```
278   -- verificar la carga de la tabla de ventas
279   SELECT
280       COUNT(*) AS CantidadVentas,
281       MIN(InvoiceDate) AS PrimeraVenta,
282       MAX(InvoiceDate) AS UltimaVenta,
283       SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) AS VentasSinFecha
284   FROM dbo.FactNikeSales;
```

	CantidadVentas	PrimeraVenta	UltimaVenta	VentasSinFecha
1	229471	2020-01-03	2024-12-30	0

Archivo `SQLQuery5.sql`

Como el dataset no trae información de clientes, se generó una dimensión de clientes ficticios a partir de las ubicaciones donde se registraron ventas. Cada cliente representa a un minorista que le compra a Nike, como una tienda deportiva o un outlet. Cada uno pertenece a una ciudad, un estado y una región, y cada venta se asigna a uno de los clientes de su misma ubicación. El script se divide en cinco partes: limpieza previa, creación de la tabla, carga de los clientes, vinculación con la tabla de ventas y verificación.

El script comienza con una limpieza previa, que permite ejecutarlo más de una vez sin errores. Con `COL_LENGTH` se consulta si la tabla de ventas ya tiene la columna `id_Client` de una ejecución anterior. Si la tiene, primero se eliminan la clave foránea `FK_Fact_Client` y el índice `IX_Fact_Client`, porque la base no permite borrar una columna que esté en uso por ellos, y después se elimina la columna. A continuación, `DROP TABLE IF EXISTS` borra la tabla `DimClient` si ya existe. Esta parte fue necesaria, por ejemplo, cuando se corrigió la carga de fechas y hubo que volver a cargar la tabla de ventas y regenerar los clientes.

SQLQuery5.sql

```sql
-- limpieza previa ------------------------------------------------------------------------------------
IF COL_LENGTH('dbo.FactNikeSales', 'id_Client') IS NOT NULL
BEGIN
    IF OBJECT_ID('dbo.FK_Fact_Client', 'F') IS NOT NULL
        ALTER TABLE dbo.FactNikeSales DROP CONSTRAINT FK_Fact_Client;

    IF EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_Fact_Client' AND object_id = OBJECT_ID('dbo.FactNikeSales'))
        DROP INDEX IX_Fact_Client ON dbo.FactNikeSales;

    ALTER TABLE dbo.FactNikeSales DROP COLUMN id_Client;
END;

DROP TABLE IF EXISTS dbo.DimClient;
GO
```

Después se crea la tabla `DimClient` con la instrucción `CREATE TABLE`. Su clave primaria (`PRIMARY KEY`) es `id_Client`, un número entero que identifica a cada cliente de forma única y que después se usa para vincularlo con las ventas. La tabla también tiene `ClientName`, con el nombre comercial del cliente, y `ClientType`, que indica si es una tienda deportiva, una tienda especializada, un outlet o una cadena minorista. Las dos columnas son de tipo `NVARCHAR`, que guarda texto, y están definidas como `NOT NULL`, es decir, no aceptan valores vacíos. Así no pueden existir clientes sin nombre ni tipo.

La ubicación de cada cliente se guarda en las columnas `id_Region`, `id_State` e `id_City`. Son claves foráneas (`FOREIGN KEY`) que apuntan a las tablas `DimRegion`, `DimState` y `DimCity`, que ya existían en el modelo. Una clave foránea obliga a que el valor exista en la tabla a la que apunta, por lo que la base no permite cargar un cliente en una ciudad, un estado o una región inexistentes. De esta forma se asegura la integridad de los datos y cada cliente queda conectado con la geografía real del dataset.

Por último, la restricción `UNIQUE` sobre `ClientName` impide que existan dos clientes con el mismo nombre. Así, en los resultados de las consultas cada nombre corresponde a un único cliente.

SQLQuery5.sql

```sql
-- crear la tabla de dimensión ------------------------------------------------------------------------
CREATE TABLE dbo.DimClient (
    id_Client       INT             PRIMARY KEY,
    ClientName      NVARCHAR(150)   NOT NULL,
    ClientType      NVARCHAR(50)    NOT NULL,

    -- ubicación del cliente (fk)
    id_Region       INT             NOT NULL,
    id_State        INT             NOT NULL,
    id_City         INT             NOT NULL,

    -- constraints
    CONSTRAINT UQ_Client_Name   UNIQUE (ClientName),
    CONSTRAINT FK_Client_Region FOREIGN KEY (id_Region)     REFERENCES dbo.DimRegion(id_Region),
    CONSTRAINT FK_Client_State  FOREIGN KEY (id_State)      REFERENCES dbo.DimState(id_State),
    CONSTRAINT FK_Client_City   FOREIGN KEY (id_City)       REFERENCES dbo.DimCity(id_City)
);
GO
```

La carga de los clientes se resuelve en una sola instrucción, escrita con expresiones de tabla comunes (CTE, por sus siglas en inglés). Una CTE se define con `WITH` y funciona como una tabla temporal con nombre que existe solo mientras se ejecuta la consulta. Permite dividir un proceso largo en pasos más simples, donde cada paso usa el resultado del anterior. En este caso se usan cuatro: `Ubicaciones`, `UbicacionesNumeradas`, `NombresComerciales` y `ClientesPorUbicacion`.

La primera, `Ubicaciones`, identifica dónde se registraron ventas. Con `SELECT DISTINCT` toma de la tabla de ventas las combinaciones distintas de región, estado y ciudad, que son 54, y con dos `INNER JOIN` les suma los nombres de la ciudad y del estado desde `DimCity` y `DimState`. La ubicación se define con las tres claves y no solo con la ciudad porque el dataset tiene ciudades con el mismo nombre en estados distintos. Es el caso de Portland, en Maine y en Oregon, y de Charleston, en Carolina del Sur y en Virginia Occidental. Si se usara solo la ciudad, esos lugares se mezclarían.

SQLQuery5.sql

```
-- cargar los clientes ficticios ------------------------------------------------------------------------------
WITH Ubicaciones AS (
    SELECT DISTINCT
        f.id_Region,
        f.id_State,
        f.id_City,
        c.City,
        s.[State]
    FROM dbo.FactNikeSales f
    INNER JOIN dbo.DimCity c
        ON f.id_City = c.id_City
    INNER JOIN dbo.DimState s
        ON f.id_State = s.id_State
),
```

La segunda, `UbicacionesNumeradas`, le asigna a cada ubicación un número correlativo del 1 al 54 con la función `ROW_NUMBER`. El orden lo define `OVER (ORDER BY City, [State])`: alfabético por ciudad y, dentro de una misma ciudad, por estado. Así, Albany es la ubicación 1, Albuquerque la 2 y Anchorage la 3. Este orden deja contiguas a las ciudades homónimas, algo que se aprovecha más adelante para que no reciban los mismos nombres comerciales.

La tercera, `NombresComerciales`, es un catálogo fijo armado con `VALUES`, que permite escribir filas directamente en el código. Contiene nueve nombres comerciales inventados, numerados del 0 al 8, como Summit Sports, Urban Runners o Champion Athletics, cada uno con su tipo de cliente. La cuarta, `ClientesPorUbicacion`, es una tabla mínima con los valores 1, 2 y 3, que representan a los tres clientes que va a tener cada ubicación.

SQLQuery5.sql

```
UbicacionesNumeradas AS (
    SELECT
        *,
        ROW_NUMBER() OVER (ORDER BY City, [State]) AS NroUbicacion
    FROM Ubicaciones
),
NombresComerciales AS (
    SELECT * FROM (VALUES
        (0, 'Summit Sports',        'Tienda deportiva'),
        (1, 'Urban Runners',        'Tienda especializada'),
        (2, 'Victory Outlet',       'Outlet'),
        (3, 'Peak Performance',     'Tienda deportiva'),
        (4, 'Fast Track Footwear',  'Tienda especializada'),
        (5, 'Mega Sport Store',     'Cadena minorista'),
        (6, 'Champion Athletics',   'Tienda deportiva'),
        (7, 'Street Kicks',         'Tienda especializada'),
        (8, 'All Stars Retail',     'Cadena minorista')
    ) AS n (idNombre, NombreComercial, TipoCliente)
),
ClientesPorUbicacion AS (
    SELECT * FROM (VALUES (1), (2), (3)) AS k (NroCliente)
)
```

Con estos cuatro pasos, la instrucción `INSERT INTO ... SELECT` arma los clientes y los guarda en `DimClient`. Primero, el `CROSS JOIN` combina cada ubicación con cada uno de los valores de `ClientesPorUbicacion`. A diferencia de un `INNER JOIN`, un `CROSS JOIN` no usa una condición de unión, sino que forma todas las combinaciones posibles entre dos tablas. Como hay 54 ubicaciones y 3 valores, el resultado tiene 162 filas, una por cliente.

El identificador de cada cliente se calcula con la fórmula `(NroUbicacion - 1) * 3 + NroCliente`. De esta forma, los tres clientes de un mismo lugar tienen números consecutivos que no se repiten entre ubicaciones: los de Albany son el 1, el 2 y el 3, los de Albuquerque el 4, el 5 y el 6, y así sucesivamente.

El nombre comercial se elige con el `INNER JOIN` a `NombresComerciales`, que usa la fórmula `(NroUbicacion % 3) * 3 + NroCliente - 1`. El operador `%` devuelve el resto de una división, así que `NroUbicacion % 3` solo puede valer 0, 1 o 2. Según ese valor, la ubicación usa los nombres 0 a 2, 3 a 5 o 6 a 8 del catálogo. Albany, por ejemplo, es la ubicación 1, que deja resto 1, así que sus clientes reciben los nombres 3, 4 y 5: Peak Performance, Fast Track Footwear y Mega Sport Store. Como las ciudades homónimas quedaron una al lado de la otra, sus restos siempre son distintos y caen en grupos diferentes. Por eso los clientes de Portland en Oregon se llaman distinto de los de Portland en Maine.

Para terminar, `CONCAT` arma el nombre de cada cliente uniendo el nombre comercial con el de la ciudad, como en "Peak Performance Albany", y se guardan las claves de su región, su estado y su ciudad. Así, cada cliente ficticio queda asociado a una ubicación real del dataset.

SQLQuery5.sql

```
72    ClientesPorUbicacion AS (
73        SELECT * FROM (VALUES (1), (2), (3)) AS k (NroCliente)
74    )
75    INSERT INTO dbo.DimClient (id_Client, ClientName, ClientType, id_Region, id_State, id_City)
76    SELECT
77        (u.NroUbicacion - 1) * 3 + k.NroCliente AS id_Client,
78        CONCAT(n.NombreComercial, ' ', u.City) AS ClientName,
79        n.TipoCliente,
80        u.id_Region,
81        u.id_State,
82        u.id_City
83    FROM UbicacionesNumeradas u
84    CROSS JOIN ClientesPorUbicacion k
85    INNER JOIN NombresComerciales n
86        ON n.idNombre = (u.NroUbicacion % 3) * 3 + k.NroCliente - 1;
87    GO
```

Luego se vincula la tabla de hechos con la nueva dimensión, es decir, se le indica a cada venta de `FactNikeSales` qué cliente la realizó. Para eso, con `ALTER TABLE ... ADD` se agrega a la tabla de ventas la columna `id_Client`, que guarda el identificador del cliente. La columna se crea primero aceptando valores vacíos (`NULL`), porque las ventas ya cargadas todavía no tienen cliente y la base no permite agregar una columna obligatoria sin un valor para cada fila.
Después se completa esa columna con un `UPDATE`, que recorre todas las ventas y le asigna un cliente a cada una. El `UPDATE` se combina con un `INNER JOIN` a `DimClient`, que cruza cada venta con los clientes de su misma región, su mismo estado y su misma ciudad. Así, una venta registrada en Portland (Oregon) solo puede quedar asociada a un cliente de Portland (Oregon). Como cada ubicación tiene tres clientes, la última condición del `JOIN` define cuál de los tres le corresponde.

Esa condición usa el identificador de la venta, `SaleID`, y el operador `%`. El resto de dividir `SaleID` por 6 siempre es un valor entre 0 y 5, y la expresión `CASE` lo traduce en un número de cliente. Si el resto es 0, 1 o 2, la venta va al primer cliente de la ubicación; si es 3 o 4, al segundo; y si es 5, al tercero. De cada seis ventas consecutivas, tres van al primer cliente, dos al segundo y una al tercero. Así, el primer cliente recibe alrededor del 50% de las ventas de su ubicación, el segundo cerca del 33% y el tercero cerca del 17%. Este reparto desigual se buscó para que los clientes no tengan todos el mismo volumen de compras, como pasaría en la realidad, y para que el análisis por cliente muestre diferencias.

Del lado del cliente, la expresión `(id_Client - 1) % 3 + 1` indica si es el primero, el segundo o el tercero de su ubicación, ya que los tres clientes de un mismo lugar tienen identificadores consecutivos. Cada venta se une con el cliente cuyo número coincide con el que calculó el `CASE`.

Una vez que todas las ventas tienen cliente, con `ALTER COLUMN` la columna `id_Client` pasa a ser obligatoria (`NOT NULL`), para que en adelante no puedan existir ventas sin cliente. Además, se la define como clave foránea hacia `DimClient` con la restricción `FK_Fact_Client`. Con esto, la base garantiza que cada venta apunte a un cliente que realmente existe, y la relación entre ventas y clientes queda formalizada en el modelo de datos. Por último, `CREATE INDEX` crea un índice sobre esa columna.

SQLQuery5.sql

```sql
-- vincular la tabla de hechos ---------------------------------------------------------------------
ALTER TABLE dbo.FactNikeSales ADD id_Client INT NULL;
GO

UPDATE f
SET f.id_Client = cl.id_Client
FROM dbo.FactNikeSales f
INNER JOIN dbo.DimClient cl
    ON  cl.id_Region = f.id_Region
    AND cl.id_State  = f.id_State
    AND cl.id_City   = f.id_City
    AND (cl.id_Client - 1) % 3 + 1 =
        CASE
            WHEN f.SaleID % 6 < 3 THEN 1
            WHEN f.SaleID % 6 < 5 THEN 2
            ELSE 3
        END;
GO

ALTER TABLE dbo.FactNikeSales ALTER COLUMN id_Client INT NOT NULL;

ALTER TABLE dbo.FactNikeSales
    ADD CONSTRAINT FK_Fact_Client FOREIGN KEY (id_Client) REFERENCES dbo.DimClient(id_Client);

CREATE INDEX IX_Fact_Client ON dbo.FactNikeSales (id_Client);
GO
```

Al final del script se verifica la carga con una consulta que muestra la tabla `DimClient` junto con los nombres de la ciudad, el estado y la región de cada cliente. El resultado confirma que cada cliente quedó creado con su nombre, su tipo y su ubicación, y que los tres clientes de cada ciudad tienen identificadores consecutivos, como los de Albuquerque (4, 5 y 6) o los de Anchorage (7, 8 y 9).

SQLQuery5.sql

```sql
-- verificar cargas ---------------------------------------------------------------------------------

-- viendo la tabla de clientes
SELECT
    cl.id_Client,
    cl.ClientName AS Cliente,
    cl.ClientType AS TipoCliente,
    c.City AS Ciudad,
    s.[State] AS Estado,
    r.Region AS Region
FROM dbo.DimClient cl
INNER JOIN dbo.DimCity c ON cl.id_City = c.id_City
INNER JOIN dbo.DimState s ON cl.id_State = s.id_State
INNER JOIN dbo.DimRegion r ON cl.id_Region = r.id_Region
ORDER BY cl.id_Client;
```

	id_Client	Cliente	TipoCliente	Ciudad	Estado	Region
4	4	Champion Athletics Albuquerque	Tienda deportiva	Albuquerque	New Mexico	West
5	5	Street Kicks Albuquerque	Tienda especializada	Albuquerque	New Mexico	West
6	6	All Stars Retail Albuquerque	Cadena minorista	Albuquerque	New Mexico	West
7	7	Summit Sports Anchorage	Tienda deportiva	Anchorage	Alaska	West
8	8	Urban Runners Anchorage	Tienda especializada	Anchorage	Alaska	West
9	9	Victory Outlet Anchorage	Outlet	Anchorage	Alaska	West
10	10	Peak Performance Atlanta	Tienda deportiva	Atlanta	Georgia	Southeast
11	11	Fast Track Footwear Atlanta	Tienda especializada	Atlanta	Georgia	Southeast
12	12	Mega Sport Store Atlanta	Cadena minorista	Atlanta	Georgia	Southeast
13	13	Champion Athletics Baltimore	Tienda deportiva	Baltimore	Maryland	Northeast
14	14	Street Kicks Baltimore	Tienda especializada	Baltimore	Maryland	Northeast
15	15	All Stars Retail Baltimore	Cadena minorista	Baltimore	Maryland	Northeast
16	16	Summit Sports Billings	Tienda deportiva	Billings	Montana	Midwest
17	17	Urban Runners Billings	Tienda especializada	Billings	Montana	Midwest
18	18	Victory Outlet Billings	Outlet	Billings	Montana	Midwest
19	19	Peak Performance Birmingham	Tienda deportiva	Birmingham	Alabama	South
20	20	Fast Track Footwear Birmingham	Tienda especializada	Birmingham	Alabama	South
21	21	Mega Sport Store Birmingham	Cadena minorista	Birmingham	Alabama	South
22	22	Champion Athletics Boise	Tienda deportiva	Boise	Idaho	West
23	23	Street Kicks Boise	Tienda especializada	Boise	Idaho	West
24	24	All Stars Retail Boise	Cadena minorista	Boise	Idaho	West
25	25	Summit Sports Boston	Tienda deportiva	Boston	Massachu...	Northeast

Archivo `SQLQuery6.sql` (consulta solicitada)

La consulta de este archivo extrae el nombre del cliente, la fecha de la compra y el total de cada venta realizada en los últimos 30 días, contados desde la fecha de la última venta del dataset, y ordena los resultados por fecha de compra descendente.

Se usaron dos tablas: `FactNikeSales`, que registra las ventas, y `DimClient`, la dimensión de clientes creada para el ejercicio. Las dos se unen con un `INNER JOIN` por la columna `id_Client`, y así cada venta queda asociada a su cliente.

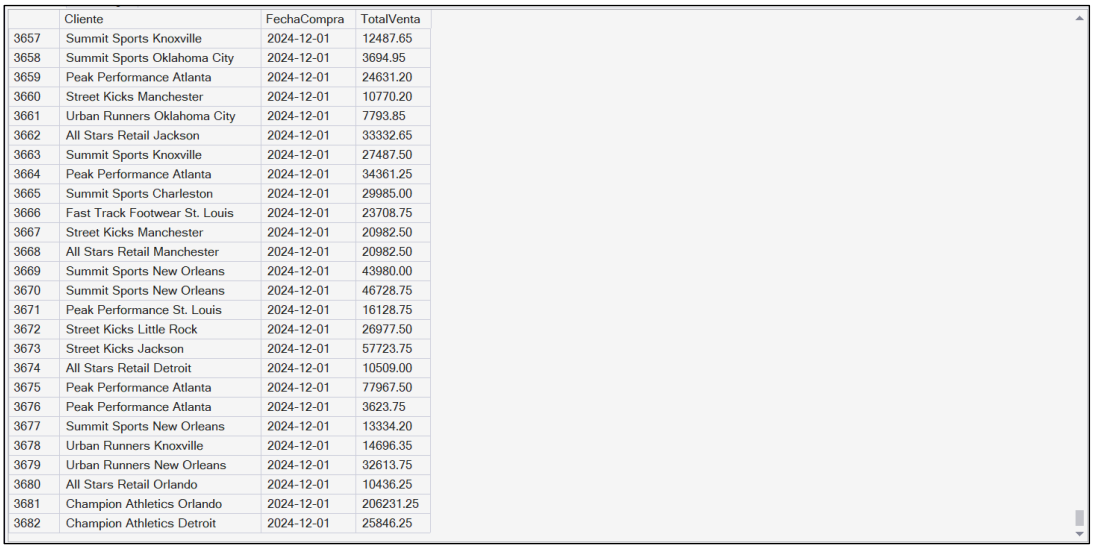
De `DimClient` se tomó `ClientName`, el nombre del cliente. De `FactNikeSales` se tomó `InvoiceDate`, la fecha de compra, que gracias a la preparación de datos está guardada con el tipo `DATE`. El total de la venta no existe como columna, así que se calculó multiplicando `PricePerUnit`, el precio por unidad, por `UnitsSold`, la cantidad de unidades vendidas. Con `AS` se les dio a las tres columnas del resultado nombres en español: `Cliente`, `FechaCompra` y `TotalVenta`.

El filtro de los últimos 30 días se resuelve en el `WHERE` y se lee de adentro hacia afuera. La subconsulta `(SELECT MAX(InvoiceDate) FROM dbo.FactNikeSales)` busca la fecha de la última venta del dataset, que es el 30 de diciembre de 2024. Luego, `DATEADD(DAY, -30, ...)` le resta 30 días a esa fecha, lo que da el 30 de noviembre de 2024. Por último, la condición `>` conserva solo las ventas posteriores a esa fecha, es decir, las del 1 al 30 de diciembre. Calcular la última fecha con una subconsulta, en lugar de escribirla a mano, hace que la consulta siga funcionando si se agregan ventas nuevas. Finalmente, `ORDER BY v.InvoiceDate DESC` ordena los resultados de la venta más reciente a la más antigua.

SQLQuery6.sql

```sql
SELECT
    cl.ClientName AS Cliente,
    v.InvoiceDate AS FechaCompra,
    v.PricePerUnit * v.UnitsSold AS TotalVenta
FROM dbo.FactNikeSales v
INNER JOIN dbo.DimClient cl
    ON v.id_Client = cl.id_Client
WHERE v.InvoiceDate > DATEADD(DAY, -30, (SELECT MAX(InvoiceDate) FROM dbo.FactNikeSales))
ORDER BY v.InvoiceDate DESC;
```

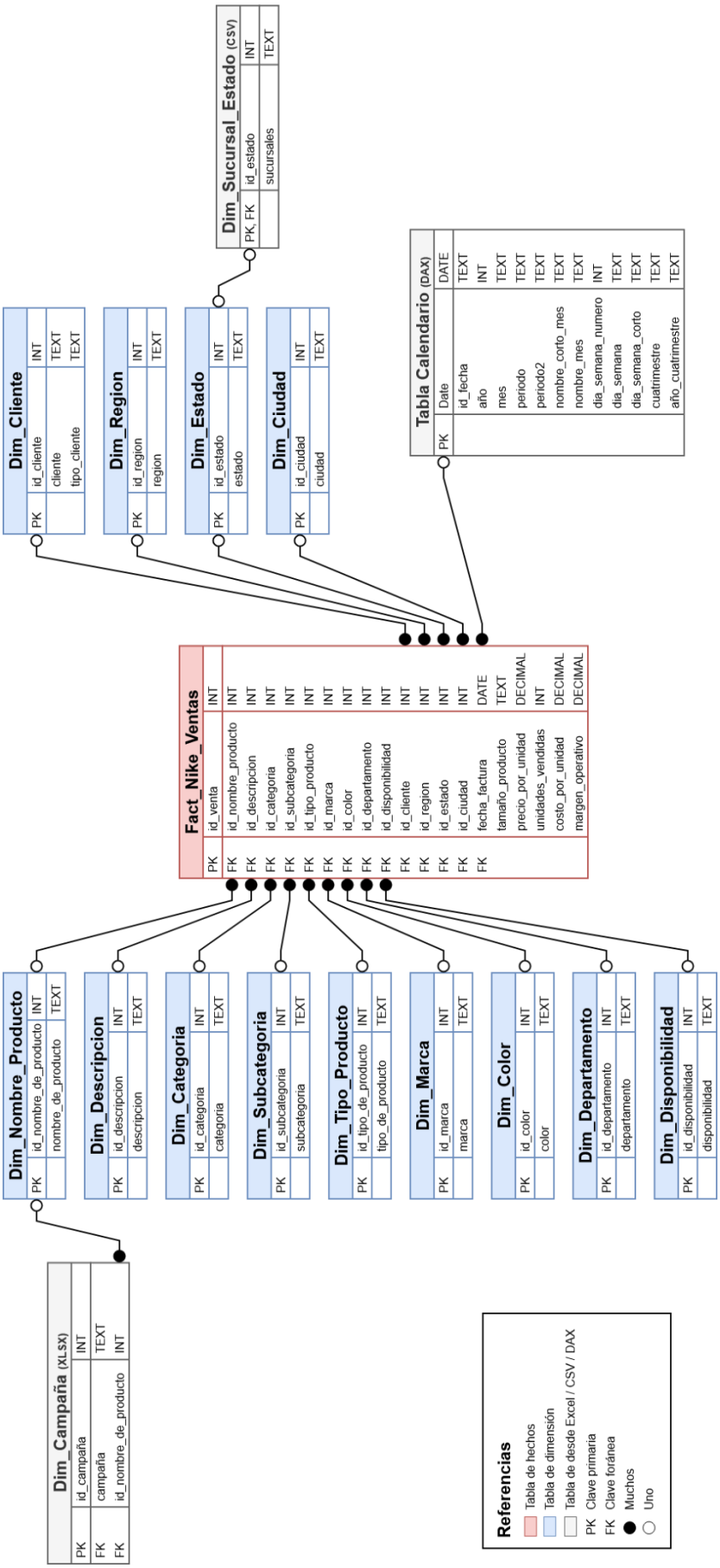
	Cliente	FechaCompra	TotalVenta
1	Summit Sports Charleston	2024-12-30	2923.05
2	Champion Athletics Orlando	2024-12-30	11200.85
3	Champion Athletics Honolulu	2024-12-30	15942.75
4	Fast Track Footwear Indianapolis	2024-12-30	16492.50
5	Fast Track Footwear Birmingham	2024-12-30	22401.70
6	Mega Sport Store Indianapolis	2024-12-30	37358.75
7	Summit Sports Wilmington	2024-12-30	13282.50
8	Summit Sports Oklahoma City	2024-12-30	24727.50
9	Peak Performance Atlanta	2024-12-30	26101.25
10	Street Kicks Dallas	2024-12-30	23718.75
11	Fast Track Footwear Burlington	2024-12-30	90588.75
12	Victory Outlet Salt Lake City	2024-12-30	1873.75
13	Peak Performance St. Louis	2024-12-30	1350.95
14	Champion Athletics Cheyenne	2024-12-30	3767.10
15	Champion Athletics Jackson	2024-12-30	3897.00
16	Fast Track Footwear Atlanta	2024-12-30	7496.25
17	Street Kicks Cheyenne	2024-12-30	12371.25
18	Victory Outlet New Orleans	2024-12-30	2236.00
19	Peak Performance Atlanta	2024-12-30	6370.75
20	Summit Sports Oklahoma City	2024-12-30	2866.65
21	Champion Athletics Cheyenne	2024-12-30	6565.50
22	Urban Runners Sioux Falls	2024-12-30	6645.25
23	Urban Runners Anchorage	2024-12-30	2935.10
24	All Stars Retail Honolulu	2024-12-30	17815.05
25	Summit Sports Boston	2024-12-30	8469.35
26	Peak Performance Burlington	2024-12-30	12754.20



	Cliente	FechaCompra	TotalVenta
3657	Summit Sports Knoxville	2024-12-01	12487.65
3658	Summit Sports Oklahoma City	2024-12-01	3694.95
3659	Peak Performance Atlanta	2024-12-01	24631.20
3660	Street Kicks Manchester	2024-12-01	10770.20
3661	Urban Runners Oklahoma City	2024-12-01	7793.85
3662	All Stars Retail Jackson	2024-12-01	33332.65
3663	Summit Sports Knoxville	2024-12-01	27487.50
3664	Peak Performance Atlanta	2024-12-01	34361.25
3665	Summit Sports Charleston	2024-12-01	29985.00
3666	Fast Track Footwear St. Louis	2024-12-01	23708.75
3667	Street Kicks Manchester	2024-12-01	20982.50
3668	All Stars Retail Manchester	2024-12-01	20982.50
3669	Summit Sports New Orleans	2024-12-01	43980.00
3670	Summit Sports New Orleans	2024-12-01	46728.75
3671	Peak Performance St. Louis	2024-12-01	16128.75
3672	Street Kicks Little Rock	2024-12-01	26977.50
3673	Street Kicks Jackson	2024-12-01	57723.75
3674	All Stars Retail Detroit	2024-12-01	10509.00
3675	Peak Performance Atlanta	2024-12-01	77967.50
3676	Peak Performance Atlanta	2024-12-01	3623.75
3677	Summit Sports New Orleans	2024-12-01	13334.20
3678	Urban Runners Knoxville	2024-12-01	14696.35
3679	Urban Runners New Orleans	2024-12-01	32613.75
3680	All Stars Retail Orlando	2024-12-01	10436.25
3681	Champion Athletics Orlando	2024-12-01	206231.25
3682	Champion Athletics Detroit	2024-12-01	25846.25

La consulta devolvió 3.682 ventas, registradas entre el 1 y el 30 de diciembre de 2024. La primera captura del resultado muestra las ventas del 30 de diciembre, la fecha más reciente, y la segunda muestra las últimas filas, del 1 de diciembre. El período comienza el 1 de diciembre porque la última venta del dataset es del 30 de diciembre de 2024, con 110 ventas ese día. No hay registros del 31 de diciembre.

2. Modelado de datos



El diagrama representa el modelo de datos del informe, organizado como un modelo estrella. En el centro está la tabla de hechos `Fact_Nike_Ventas`, que corresponde a la entidad `Ventas`. Cada fila es una venta, identificada por su clave primaria `id_venta`, y guarda la fecha, el talle, el precio por unidad, las unidades vendidas, el costo y el margen operativo. Además, contiene las claves foráneas que la conectan con las demás tablas.

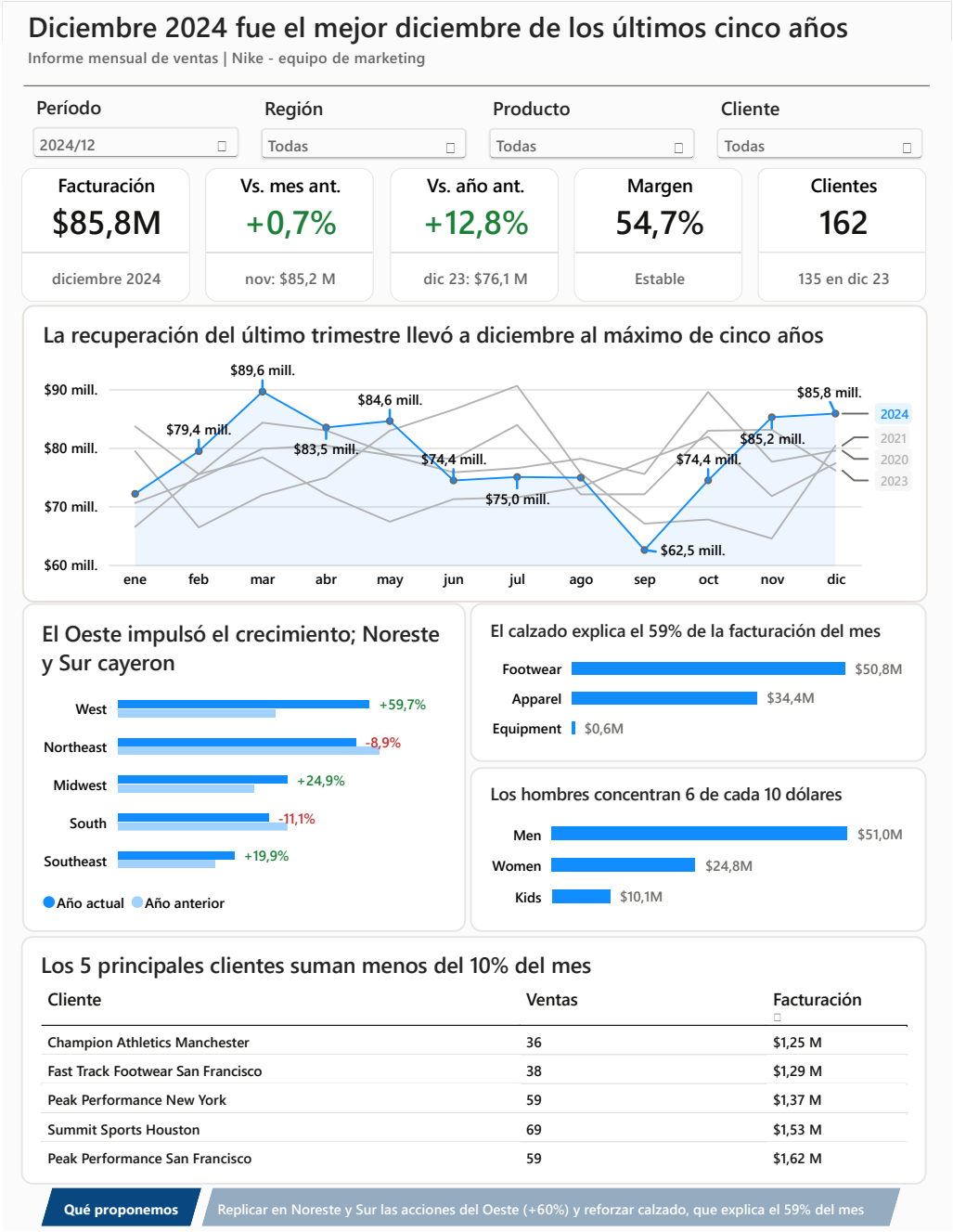
La entidad `Clientes` está representada por `Dim_Cliente`, con su clave primaria `id_cliente`, el nombre del cliente y su tipo. La relación entre Clientes y Ventas es de uno a muchos (1:N): un cliente puede realizar muchas ventas, pero cada venta pertenece a un único cliente. Esto se resuelve con la clave foránea `id_cliente` en la tabla de ventas.

La entidad Productos no está en una sola tabla, sino distribuida en varias dimensiones que describen cada característica del producto: `Dim_Nombre_Producto`, `Dim_Descripcion`, `Dim_Categoria`, `Dim_Subcategoria`, `Dim_Tipo_Producto`, `Dim_Marca`, `Dim_Color`, `Dim_Departamento` y `Dim_Disponibilidad`. Cada una tiene su propia clave primaria y se relaciona con la tabla de ventas también en una relación de uno a muchos: un mismo producto, categoría o marca aparece en muchas ventas, pero cada venta corresponde a un único valor de cada una. Separar los atributos del producto en dimensiones evita repetir textos en cada venta y facilita filtrar el análisis por cualquiera de ellos.

El modelo se completa con las dimensiones geográficas (`Dim_Region`, `Dim_Estado` y `Dim_Ciudad`), que indican dónde se realizó cada venta, y con la `Tabla Calendario`, creada con DAX, que se relaciona con la fecha de la venta y permite analizar por año, mes o período. En el diagrama, el círculo vacío indica el lado “uno” de cada relación y el círculo lleno el lado “muchos”.

3. Visualización y storytelling

Decidí utilizar Power BI como herramienta ya que fue la aprendida en el curso. Armé un informe mensual de ventas pensado para una presentación simulada ante un equipo de marketing. Mi objetivo fue que cualquier persona del equipo pudiera entender en pocos segundos cómo le fue al negocio en el mes, por qué y qué conviene hacer a continuación. Por eso organicé el dashboard como un relato que va de lo general a lo particular y termina en una recomendación concreta, en lugar de presentar gráficos sueltos:



El informe se abre con un título que funciona como conclusión principal: "**Diciembre 2024 fue el mejor diciembre de los últimos cinco años**". Elegí un título afirmativo y no descriptivo porque le comunica al lector el mensaje central antes de que mire cualquier gráfico. Con el mismo criterio, cada gráfico tiene un título que expresa su conclusión y no solo lo que muestra, para que el equipo de marketing pueda entender el informe leyendo únicamente los títulos. Debajo ubiqué **cuatro segmentadores** (período, región, producto y cliente) para que el equipo pueda explorar los datos y repetir el análisis en otros meses o mercados.

A continuación incluí **cinco tarjetas de indicadores que resumen el mes** en sus números clave. Diciembre facturó 85,8 millones de dólares, un 0,7% más que noviembre y un 12,8% más que diciembre de 2023, cuando se habían facturado 76,1 millones. El margen se mantuvo estable en 54,7% y 162 clientes realizaron compras, frente a 135 un año antes. Usé tarjetas porque son la forma más directa de mostrar valores puntuales. Además, pinté las variaciones de verde o rojo según sean positivas o negativas, para que el resultado se lea a simple vista.

El segundo bloque responde a la pregunta de cómo se llegó a ese resultado. Para eso elegí un gráfico de líneas, titulado "**La recuperación del último trimestre llevó a diciembre al máximo de cinco años**", con la facturación mensual de 2024 destacada en azul y los años anteriores en gris como referencia. El gráfico de líneas es el más adecuado para mostrar tendencias en el tiempo, y el contraste de colores permite comparar sin recargar la vista. Ahí se observa que 2024 no fue un año lineal: tuvo un pico en marzo (89,6 millones), una caída fuerte en septiembre (62,5 millones, el peor mes del año) y una recuperación sostenida en el último trimestre, que llevó a diciembre por encima de todos los años anteriores. Esto muestra que la caída de septiembre fue puntual y no un cambio de rumbo.

Luego profundicé en dónde se ganó y dónde se perdió con un gráfico de barras agrupadas, titulado "**El Oeste impulsó el crecimiento; Noreste y Sur cayeron**". El gráfico compara cada región en diciembre de 2024 contra diciembre de 2023 y muestra el porcentaje de variación al costado. Elegí barras horizontales porque facilitan comparar magnitudes entre categorías y permiten leer los nombres sin rotarlos. Este gráfico revela que el crecimiento fue desigual. El Oeste fue el motor del mes, con un aumento del 59,7%, y el Medio Oeste y el Sudeste también crecieron (24,9% y 19,9%). En cambio, el Noreste cayó un 8,9% y el Sur un 11,1%. El caso del Noreste es una señal de alerta, porque en 2023 era la región líder y en 2024 perdió ese lugar.

Al lado ubiqué dos gráficos de barras que muestran qué se vende y a quién. El primero, "**El calzado explica el 59% de la facturación del mes**", muestra que el calzado aportó 50,8 millones, seguido por la ropa (34,4 millones), mientras que el equipamiento es marginal. El segundo, "**Los hombres concentran 6 de cada 10 dólares**", muestra que los hombres concentran la mayor parte de las ventas (51,0 millones), frente a las mujeres (24,8 millones) y los niños (10,1 millones), lo que indica que estos dos segmentos tienen espacio para crecer.

Para bajar el análisis al nivel más concreto, agregué una tabla con los cinco clientes de mayor facturación del mes, titulada "**Los 5 principales clientes suman menos del 10% del mes**". Elegí una tabla porque, para pocos registros con nombre propio, permite leer valores exactos mejor que un gráfico. Encabeza la lista "Peak Performance San Francisco", con 1,62 millones. Los cinco clientes juntos representan menos del 10% de la facturación, lo que indica que el negocio no depende de unos pocos compradores.

Por último, cerré el informe con un recuadro de **propuesta**, porque una presentación ante un equipo de marketing tiene que terminar en una acción y no solo en un diagnóstico. Propongo replicar en el Noreste y el Sur las acciones comerciales que funcionaron en el Oeste, y reforzar el calzado, que ya es la categoría principal.

La conclusión del relato es que Nike cerró 2024 con su mejor diciembre en cinco años, con más clientes y sin resignar margen, y que el desafío para el año siguiente no es solo crecer, sino lograr que ese crecimiento llegue de forma pareja a todas las regiones.